

第十九章 让大模型学会动手：Agent

19.1 Function Calling：从回答问题到完成任务

你对 AI 助手说“帮我查一下北京今天天气怎么样”，过了几秒钟，它告诉你“北京今天晴，22 度，北风 3 级”。你问它“387 乘以 529 等于多少”，它给你一个精确到个位数的答案。你让它“帮我搜一下最近有什么好看的科幻片”，它真的列出了几部最近上映的电影。

但你一路读到这里，已经知道了：大模型从头到尾只干一件事——根据前面的文字，预测下一个词。它的全部能力就是生成文本。

那它怎么查的天气？怎么算的乘法？怎么搜的电影？

答案是：它没有查，也没有算，也没有搜。这些事情全是别人帮它干的。

19.1.1 模型只是“说了一句话”

我们把刚才那个查天气的过程放慢来看。你在聊天框里打了“北京今天天气怎么样”，点了发送。然后你看到屏幕上出现了“北京今天晴，22 度”这样的回复。从你的视角看，好像模型自己跑出去查了天气。

但实际上发生的事情是这样的：

模型看到你的问题之后，它知道自己回答不了：天气是实时数据，它的训练数据里不可能有今天的天气。但它知道有一个天气查询的工具可以用（因为开发者提前告诉它了）。于是模型做了一个决定：我不直接回答用户，我先告诉程序“帮我去查一下北京的天气”。

注意这里的关键：模型这时候的回复，不是说给你（用户）听的，而是说给它背后的程序听的。你在屏幕上看到的可能是一个“正在查询中”的加载动画，而模型在后台其实输出了一段结构化的文本，大概意思是：“我要调用天气查询工具，参数是北京。”

然后，开发者写的后台程序收到了这段文本，解析出“要查天气”和“查的是北京”这两个信息，然后替模型去调用了真正的天气服务，拿到了“晴，22 度，北风 3 级”这个结果。程序再把这个结果塞回给模型，告诉它：“你要的天气数据来了，结果是这些。”

模型拿到结果之后，这才写出了那句“北京今天晴，22 度，北风 3 级”——这一次的回复，才是说给你看的。

整个过程里，模型做的事情只有两件：第一件，说了一句“帮我查天气”（这句话是说给程序听的）；第二件，看到查询结果之后，用自然语言把结果告诉你（这句话是说给你听的）。至于真的去联网查天气这个动作，模型完全没有参与。它从始至终都坐在那里生成文本，一步也没挪过。

这个机制就叫 Function Calling，也有人叫 Tool Use（工具调用）。名字里虽然有“调用”两个字，但模型本身并没有调用任何东西。它只是生成了一段话表达“我想用

这个工具”的意图，真正去执行的是外面的程序。

Function Calling 这个名字翻译成中文就是“函数调用”。但你现在已经知道了，模型并没有调用任何函数。叫“Function Requesting”或者“Tool Suggesting”可能更贴切，但名字已经叫开了，全行业都这么说，你也就这么用吧。

19.1.2 最原始的做法：靠 Prompt 硬挤，靠正则硬解

Function Calling 现在是各家大模型 API 的标准功能。但在这个标准出现之前，工程师们就已经在想办法让模型“用工具”了。

做法很朴素：在 System Prompt 里写一段话，告诉模型“如果用户问天气，你就输出一段 JSON，里面包含工具名和参数”。然后在后台代码里，拿到模型的回复后，试着从里面把 JSON 抠出来，解析参数，去调真正的接口。

这条路能走通，但三个问题会让你很头疼。

第一，模型经常忍不住在 JSON 前面加一句废话。你让它只输出 JSON，它偏偏先来一句“好的，我来帮你查一下天气哦~”，然后才是 JSON。你的代码期待的是纯 JSON，前面多了一句话就解析报错了。你被迫写各种正则表达式去剥掉这些多余的内容，但你永远猜不到模型下次会加什么。

第二，参数名一会儿一变。你在 Prompt 里规定参数叫city，模型有时候确实写city，有时候写location，有时候写city_name。你的代码按city去取值，取到空的，整条链路就断了。

第三，模型搞不清楚什么时候该用工具。用户问“北京今天多少度”，它应该输出 JSON。用户问“你觉得北京好玩吗”，这是闲聊，不需要工具，但模型有时候也给你输出一个 JSON。反过来，有些该用工具的场景，它又选择自己编一个答案。

这三个问题的根源是同一件事：靠 Prompt 来约束模型的输出格式，是“软约束”。大多数时候它会遵守，但你没办法保证它每次都遵守。前面讲结构化输出的时候，你已经见识过这种不靠谱了。

Function Calling 就是对这套土方法的正规化升级。

19.1.3 开发者给模型递了一份“工具说明书”

在正式走完整流程之前，有一件事你要先搞明白：模型怎么知道自己有工具可以用？

答案是：你告诉它的。

你在调用大模型 API 的时候，除了传用户的消息（“北京今天天气怎么样”），还会额外传一份“工具清单”。这份清单描述了你准备给模型用的每一个工具：工具叫什么名字、是干什么的、需要哪些参数、每个参数是什么类型。

比如，你给模型准备了一个天气查询工具。你传过去的工具描述大概长这样（这是一个简化后的 JSON 示意，不是某家 API 的完整字段；重点看它在表达什么信息）：

```
{
  "name": "get_weather",
  "description": " 查询指定城市的实时天气",
  "parameters": {
    "city": " 必填, 城市名, 例如北京",
    "unit": " 可选, celsius (摄氏度) 或 fahrenheit (华氏度) "
  }
}
```

这段 JSON 在说什么? 翻译成大白话就是: 有一个工具叫 `get_weather`, 它的功能是“查询指定城市的实时天气信息”。它需要两个参数: `city` 是必填的, 告诉工具你要查哪个城市; `unit` 是可选的, 不填就用默认值。 `unit` 只有两个选项可以填, 要么是 `celsius` (摄氏度), 要么是 `fahrenheit` (华氏度), 不能随便写别的。

真实 API 里的工具描述会更严格, 通常会写成 JSON Schema, 用 `type`、`properties`、`required` 这些字段把参数类型、可选值和必填项写清楚。这里先不把那套完整格式展开, 免得你第一眼就被字段名淹了。你现在只要抓住核心: 开发者把“工具叫什么、能干什么、需要哪些参数”告诉模型, 模型才知道自己有什么工具可以用。

这份工具描述, 就是你给模型的一份“使用说明书”。模型拿到这份说明书, 才知道自己有什么工具可以用、每个工具需要什么参数。如果你什么都不传, 模型根本不知道有工具这回事, 它只会正常回答问题。

那模型最终看到的完整输入是什么样的? 大模型 API 的框架会把你传过去的工具清单和用户消息拼在一起, 变成一大段文本送进模型。模型真正看到的東西, 大致相当于这样:

【系统提示】

你是一个有用的助手。你可以使用以下工具来帮助回答用户的问题:

工具: `get_weather`

功能: 查询指定城市的实时天气信息

参数:

`city` (必填, 字符串): 要查询的城市名称, 例如北京、上海

`unit` (可选, 字符串, 可选值: `celsius` (摄氏度) / `fahrenheit` (华氏度)): 温度单位

当你需要使用工具时, 请输出工具调用的结构化信息。如果不需要使用工具, 直接回答用户的问题。

【用户消息】

北京今天多少度?

如果换成你在代码里调用 API 时更接近真实的写法, 大概会长这样:

```
messages = [
    {"role": "system", "content": " 你是一个有用的助手。"},
    {"role": "user", "content": " 北京今天多少度？"}
]

tools = [
    {
        "name": "get_weather",
        "description": " 查询指定城市的实时天气",
        "parameters": {
            "city": " 必填，城市名，例如北京",
            "unit": " 可选，celsius（摄氏度）或 fahrenheit（华氏度）"
        }
    }
]
```

也就是说，开发者真正传给 API 的，通常是两块东西：一块是对话消息，另一块是工具清单。API 框架会在内部把它们整理成模型能理解的上下文。不同厂商整理方式不一样，所以这里不追求复刻某一家 API 的完整格式，只让你看清楚：用户消息是一块，工具清单是另一块。

前面两种写法分别对应两个视角：自然语言版帮你理解模型看到的大意，代码版帮你理解开发者实际传给 API 的结构。无论具体格式怎么变，关键都是同一件事：模型同时看见用户消息和工具清单，然后判断是直接回答，还是先调用工具。

实战避坑 · 不同模型厂商的 Function Calling 格式不通用

OpenAI 的 `tool_calls` 格式、Anthropic 的 Tool Use 格式、开源模型（比如 Qwen、GLM）各自定义的格式，长得都不一样。字段名不同、嵌套结构不同、甚至“模型决定不调用工具”时返回的内容也不同。你照着一家的教程写好的解析代码，换一个模型直接报错。更隐蔽的是，有些开源模型声称“兼容 OpenAI 格式”，但在边界情况下（比如模型同时调用两个工具、或者模型决定不调工具时）返回的结构和 OpenAI 有细微差异，你的代码在正常情况下能跑通，在边界情况下就炸了。新人最常犯的错是：拿着某一家的文档当标准来学，换模型的时候以为只要改个 API 地址就行，结果调了一下午的 bug。养成习惯：每换一个模型，先去看它自己的 Function Calling 文档，特别是看返回结构的示例，不要假设和之前用的那家一样。

19.1.4 一次完整的工具调用，五步走通

现在你知道了模型看到的输入长什么样了。我们接着用“北京今天多少度”这个例子，一步一步走完整个流程。

第一步：你把工具清单和用户消息一起传给模型。

这一步刚才已经讲了。你在代码里调用大模型 API，传了两样东西：用户的消息“北京今天多少度”和天气查询工具的描述。API 框架把它们拼成一大段文本，送进模型。

这一步是你（开发者）在做的事情，用户和模型都还没有出场。

第二步：模型判断需要调用工具，输出一段“调用意图”。注意，这段输出不是给

用户看的。

模型看到用户问的是今天的天气。它在训练的时候见过很多类似的场景，学会了一个判断逻辑：实时天气数据我自己不知道，但我有一个天气查询工具可以用，我应该用它。

这里要注意：这个判断逻辑是训练样本教出来的，模型天生并不知道什么时候该用工具。如果训练数据里很少出现“遇到实时天气问题，就调用天气工具”这种例子，模型可能明明有工具清单，也想不起来用。反过来，如果训练数据把工具调用教得太泛，模型又可能变成“工具上瘾”：连“1+1 等于几”这种自己能回答的问题，也想先去调工具。所以 Function Calling 训练里很微妙的一点是：模型既要学会用工具，也要学会什么时候不用工具。

于是模型做了一个重要的动作：它没有直接回答用户，而是输出了一段结构化信息。这段信息的核心内容是：

```
我要调用 get_weather 这个工具
参数: city = " 北京", unit = "celsius"
```

实际的格式会多一些固定的字段（比如一个id用来标识这次调用），但核心就是这两样：工具名和参数值。

这里有一件非常重要的事情，我要反复强调：模型这一次的输出，不是给用户看的。用户在屏幕上看不到这段内容。这段输出是给你的后台程序看的，你的程序需要解析这段结构化信息，从中提取出“要调用什么工具”和“参数是什么”，然后替模型去办事。

你可以这样理解：模型有两种回复模式。一种是正常模式，它生成一段自然语言，这段话会直接显示在用户的聊天界面上。另一种是工具调用模式，它生成一段结构化的调用意图，这段话不会显示给用户看，只有你的程序能看到。在第二种模式下，用户在屏幕上看到的可能是一个旋转的加载图标，或者一行小字“正在查询天气……”。

模型是怎么做出“要用工具”还是直接回答这个判断的？本质上还是在做 token 生成。它并没有运行什么特殊的决策程序。它在训练阶段（主要是指令微调阶段）见过大量类似的例子：当用户问实时数据、而工具清单里又有合适的工具时，训练数据里的标准答案就是输出调用意图。模型从这些样本里学会了这种行为，就像它学会了“用中文回答中文问题”一样。

第三步：你的程序拿到调用意图，去真正执行。

你的后台程序收到了模型输出的调用意图，解析出两个信息：工具名是get_weather，参数是city="北京"和unit="celsius"。

接下来你的程序要做的事情，就是去调用真正的天气服务。这里我要插一句，因为有些读者可能会问：凭什么你的程序能查到天气？

你的程序能查到天气，是因为市面上有天气数据服务商（比如和风天气、Open-WeatherMap）开放了数据接口：你按照它规定的格式发一个网络请求过去，它就把数据返回给你。这种“通过网络请求获取数据”的标准方式叫 API（Application Programming Interface）。你平时用的手机天气 App，背后做的也是同一件事。

所以你的程序做的事情是：按照天气服务的 API 规定，发一个网络请求过去，带上“北京”和“摄氏度”这两个参数。天气服务处理完之后，返回了这样的数据：

```
{"temperature": 22, "condition": " 晴", "wind": " 北风 3 级"}
```

这一步和大模型没有任何关系。模型不知道你有没有去查，不知道查没查成功，不知道结果是什么。它在等你把结果送回来。

顺便说一句，如果这一步出了问题呢？比如天气服务挂了，或者网络请求超时了，你的程序没能拿到天气数据——那怎么办？你不能假装这一步没发生，你需要把“查询失败了”这个信息也告诉模型。模型看到失败信息后，通常能理解情况，它会回复用户类似于“抱歉，天气服务暂时无法访问，请你稍后再试”这样的话。所以不管工具调用成功还是失败，你都需要把结果反馈给模型。

第四步：把工具执行的结果塞回给模型。

你拿到了天气数据，现在要把它送回给模型。做法是把这个结果追加到对话历史里，作为一条新的消息，再次调用模型 API。

这时候模型看到的完整上下文变成了什么呢？让我用自然语言把它展开写出来：

【系统提示】

你是一个有用的助手。你可以使用以下工具……（和第一步一样的工具描述，这里省略）

【用户消息】

北京今天多少度？

【助手的上一轮输出】

我要调用 `get_weather` 工具，参数是 `city="北京", unit="celsius"`

【工具返回的结果】

```
{"temperature": 22, "condition": "晴", "wind": "北风3级"}
```

模型现在看到了完整的四段信息：系统提示和工具描述、用户的问题、自己之前做的调用决定、以及工具返回的结果。它的任务是：根据工具返回的数据，生成一段自然语言来回复用户。

你注意到了吗？模型被调用了两次。第一次是在第二步，模型看到用户的问题和工具清单，输出了调用意图。第二次是现在，模型看到了工具返回的结果，要生成最终回答。中间的去查天气，是你的程序在做的事情，模型完全不参与。

第五步：模型根据工具结果，生成最终回答——这次是给用户看的。

模型看完了上面那四段上下文，输出了最终回答：

北京今天天气晴，气温 22°C，北风 3 级。

这一次的输出，是说给用户看的。用户在屏幕上看到的就是这句话。它和第二步那段给程序看的调用意图完全不同：第二步的输出是结构化数据，只有程序能解析；第五步的输出是自然语言，是直接展示在聊天界面上的。

到这里，一次完整的工具调用就结束了。

回头看一遍整个流程：你传入工具清单和用户消息 → 模型输出调用意图（给程序看的）→ 你的程序执行工具 → 你把结果塞回给模型 → 模型生成最终回答（给用户看的）。五步，两次模型调用，中间一次程序代劳。这个顺序一定要记住，后面排查工具调用问题时，基本就是沿着这条链路一段一段往回查。

有一件事我再多说一句：整个过程里，模型可以触发真实世界的操作。查天气还好，最多查不到。但如果你给模型挂的工具是“发邮件”或“删除订单”，模型填错了参数，你的程序就会真的发错邮件或者删错数据。所以在第三步，也就是你的程序真正执行工具之前，做必要的参数校验和权限控制是很重要的。不能无条件信任模型给出的参数，就像你不会无条件信任一个实习生填的审批单一样。

面试考点 · 为什么工具调用后还要再问模型一次

常见问法: “Function Calling 里, 模型输出了工具调用意图之后, 为什么还要把工具结果塞回去, 再调用一次模型?”

典型错误: 回答 “因为模型不能联网”。这只解释了为什么需要外部程序帮忙, 没解释为什么工具执行完之后还要再问模型一次。

正确思路: 模型在输出工具调用意图时, 还不知道工具的执行结果是什么。它不知道北京今天到底是 22 度还是 35 度。没有这个结果, 它就没法生成 “北京今天 22 度” 这句最终回答。所以必须先让它输出调用意图, 等外部程序执行完拿到结果, 再把结果塞进上下文, 让模型基于新信息生成最终回答。本质上是一个信息依赖的问题: 后一次调用的输入, 依赖于前一次调用触发的外部执行结果。模型的一次推理是一个完整的生成过程, 它不能 “暂停在中间等外部数据到了再继续”。生成结束就是结束了, 要想让模型看到新信息, 只能把新信息拼进上下文, 重新发起一次调用。边界也要说清楚: 如果用户的问题不需要工具, 模型一次调用就能直接回答; 如果只调用一个工具, 通常是 “先输出调用意图, 再根据工具结果回答” 这两次; 如果需要连续调用多个工具, 就可能来回多轮。后面讲 ReAct 和 Agent 时, 你会看到这种循环变得更明显。

19.1.5 再来一个例子: 计算器

天气查询的例子需要调用外部的 API, 涉及网络请求。为了让你更清楚地看到 Function Calling 的本质, 我们再看一个更简单的例子: 计算器。

大模型做数学计算是出了名的不靠谱。你问它 “387 乘以 529 等于多少”, 它可能给你一个看起来很像那么回事但其实是错的数字。原因你已经知道了——模型在做的是 “预测下一个 token”, 不是在算数学。它从训练数据里学到了一些数学的模式, 但涉及到大数字的精确运算, 它经常出错。

但你实际用 ChatGPT 或者 Kimi 的时候, 问它这种计算题, 它几乎每次都能答对。怎么回事?

和查天气一模一样。模型自己没有算, 它调用了计算器。

这个 “计算器” 甚至不需要是什么外部服务。它可以就是一个最简单的 Python 函数:

```
1 def calculate(expression):  
2     return eval(expression)
```

就这么两行代码, 接收一个数学表达式 (比如 “387 * 529”), 用 Python 算出结果返回。

实战避坑 · 不要在线上对用户输入使用 eval

这里为了让例子最短，故意写成了 `eval`。真实项目里不要这样做。`eval` 会执行传入的任何 Python 表达式，如果用户通过对话诱导模型在 `expression` 参数里填入恶意代码，你的服务器就会真的去执行它。正确做法是用白名单表达式解析器，或者只开放加减乘除这类明确允许的运算。这和前面讲的“不能无条件信任模型给出的参数”是同一件事：模型填的参数内容不可控，你的程序必须在执行前做校验和过滤。

你把这个函数作为一个工具告诉模型，工具描述大致是：

工具名：`calculate`

功能：计算一个数学表达式的结果，支持加减乘除和常见数学运算

参数：`expression`（必填，字符串）：需要计算的数学表达式，例如 `"387 * 529"`

用户问“387 乘以 529 等于多少”。模型看到这个问题，又看到自己有一个计算工具可以用。它知道自己算大数字不准，但计算器算得准，于是它做了和查天气时一样的事：输出一段调用意图（给程序看的，不是给用户看的），说的是“我要调用 `calculate` 工具，参数是 `expression='387 * 529'`”。

你的程序收到这个意图，执行 `calculate("387 * 529")`，Python 算出结果是 204723。你的程序把这个结果塞回给模型。模型看到结果后，生成最终回答：

$387 \times 529 = 204,723$

用户看到的是一个精确的答案，以为模型算术能力很强。但实际上模型一步计算都没做，它只是在正确的时机说了一句“帮我算一下 387 乘以 529”，然后用 Python 算出来的结果回答了用户。

这个例子能帮你看清 Function Calling 最本质的东西：模型的角色是“决策者”，不是“执行者”。它决定什么时候需要工具、用哪个工具、参数怎么填，但真正干活的是外面的程序。程序可以是一个网络请求（查天气），也可以是一个两行的 Python 函数（算数学），也可以是一条数据库查询语句——什么都行。模型不关心工具内部是怎么实现的，它只需要知道工具叫什么、做什么、需要什么参数。

19.1.6 工具描述写得好不好，差别有多大

模型选对工具、填对参数的前提是什么？是你给它的工具描述足够清楚。

这和写 Prompt 是一回事。你在前面学过的那些写好 Prompt 的技巧，比如说清楚任务、给示例、标明边界，在写工具描述的时候同样有用。

我用一个例子来说明工具描述的好坏差多远。

假设你在做一个电商的 AI 客服，你有一个商品搜索的工具。下面是两种不同的描述：

描述 A：“搜索商品”

描述 B：“根据用户描述的购物需求，在商品数据库中搜索匹配的商品。支持按品类、品牌、价格范围筛选。当用户表达购物意图、询问某类商品是否有售、或想要寻找特定条件的商品时使用”

用户说：“有没有 500 块以内的蓝牙降噪耳机？”

用描述 A 的话, 模型看到“搜索商品”四个字, 它可能会犹豫: 用户说的是“有没有”, 这算不算“搜索”? 结果它可能选择不调用工具, 自己凭训练数据编了一个回答——推荐了几款耳机, 但价格和型号都是它瞎编的, 用户拿着去搜发现根本不存在。

用描述 B 的话, 模型看到“询问某类商品是否有售”“按价格范围筛选”, 和用户的问题完美匹配。它会果断调用工具, 传入“蓝牙降噪耳机”“500 元以内”这些参数。工具返回真实的商品数据, 用户拿到的是准确的信息。

同一个用户问题, 因为工具描述不一样, 模型做出了完全不同的决策。一个导致用户看到了编造的信息, 另一个给了用户真实有用的数据。

如果你的工具清单里有好几个工具, 描述就更重要了。假设你同时有“搜索商品”和“查看商品详情”两个工具。“搜索”是根据关键词从数据库里找商品, “查看详情”是根据一个具体的商品编号去看它的详细参数。如果这两个工具的描述都写得很模糊, 比如一个写“搜索商品”, 一个写“查询商品信息”, 模型就很容易搞混。用户说“这款 AirPods Pro 的续航怎么样”, 这是需要查看特定商品详情的场景, 但模型可能误用了搜索工具, 搜出来一堆不相关的耳机列表。

解决办法就是把每个工具的职责边界写清楚。“搜索商品”的描述里写明“根据用户的模糊需求搜索, 适用于用户不知道具体要哪款产品的场景”; “查看商品详情”的描述里写明“根据产品名称或编号查看特定商品的详细参数, 适用于用户已经有明确目标的场景”。边界清楚了, 模型犯错的概率就小得多。

参数的描述也一样。如果你有一个 `date` 参数, 只写了类型是“字符串”, 没有任何其他说明, 模型可能填“明天”、“下周一”、“3 月 5 号”, 格式五花八门, 你的程序根本没法解析。但如果你在参数描述里写清楚“格式为 YYYY-MM-DD, 例如 2024-03-05”, 模型就会按这个格式来填。

你别觉得描述随便写写就行。如果工具描述写得太笼统, 模型在 20% 的请求里选错了工具, 直到用户投诉量涨起来才发现问题。排查的时候你要一条一条翻调用日志, 看模型在哪些 case 上做了错误的选择, 然后回过头去改工具描述、加 System Prompt 约束、重新测试——前前后后折腾大半天, 就因为当初写描述的时候偷了十分钟的懒。

19.1.7 模型什么时候用工具, 什么时候不用

你把工具清单传给了模型, 不代表模型每次都会用。

用户问“北京今天多少度”——需要实时数据, 模型调工具。用户问“你觉得秋天适合去北京玩吗”——这是一个常识性的问题, 模型自己就能回答, 不需要工具。模型会自己判断“这个问题我能不能自己答”, 如果能, 它就跳过工具, 直接生成自然语言回复给用户。

这个判断大多数时候是准的, 但不是每次都准。有两种出错的方向:

一种是“该用工具但没用”。用户明明在问实时数据, 模型却选择自己编了一个答案。遇到这种情况, 你首先检查工具描述是不是写得不够清楚, 模型可能没有意识到这个问题和工具是匹配的。其次你可以在 System Prompt 里加一句提示, 比如“当用户询问实时信息时, 请优先使用工具查询, 不要自行编造数据”。

另一种是“不该用工具但用了”。用户在闲聊, 模型却莫名其妙地调用了工具。这种情况相对少见, 通常也是因为工具描述写得太宽泛, 覆盖范围太广, 导致模型把很多问题都往工具上靠。解决方法是把工具描述的适用场景写得更精确。

还有第三种出错方式: 用对了工具, 但参数填错了。

这个错误在测试阶段很难发现——你测的那几个 case 可能恰好参数都是对的。上线之后遇到千奇百怪的用户输入，模型开始编造参数了，你的程序拿着一个不存在的经纬度去查餐厅，返回空结果，用户看到的是“附近没有餐厅”。如果你没有加参数校验，这个 bug 可能悄悄跑了好几天，直到有人在群里吐槽“你们这个 AI 是不是瞎的”你才知道出事了。

你可能会注意到，不管哪种出错方向，根源都是工具描述的问题。所以花时间打磨工具描述，和花时间打磨 Prompt 一样，是非常值得的投入。

19.1.8 多个工具和多次调用

真实项目里，你传给模型的工具通常不止一个。可能有查天气的、查订单的、搜索商品的、计算数字的……模型需要从这些工具里挑出正确的那一个，或者判断一个都不用。

更复杂的场景是，一个用户问题需要连着调好几个工具。比如用户说“帮我查一下北京明天的天气，然后根据天气推荐一个适合的户外活动”。这里需要先查天气，拿到结果之后，再根据天气情况去查活动推荐。第二个工具的参数依赖于第一个工具的返回结果，所以只能一个一个来——先调第一个，结果塞回模型，模型再决定怎么调第二个，你的程序再去执行，结果再塞回来……

当这种“调用 → 拿结果 → 根据结果决定下一步 → 再调用”的循环变成模型的主要工作模式时，它就不再是简单的工具调用了，而是进入了 Agent（智能体）的范畴。Function Calling 解决的是“一次工具调用的标准协议”：选什么工具、填什么参数、结果怎么传回来。Agent 解决的是“自主决策调几次、调哪些、什么时候停”的问题。Function Calling 是 Agent 最重要的基础零件，但两者不是同一件事。这部分内容在后面讲 Agent 的时候会完整展开。

面试考点 · Function Calling 和 Agent 的边界

常见问法：“Function Calling 和 Agent 有什么区别？有了 Function Calling 是不是就等于有了 Agent？”

典型错误：把两者混为一谈，认为“模型能调用工具就是 Agent 了”。

正确思路：Function Calling 解决的是单次工具调用的标准协议——选什么工具、填什么参数、结果怎么回传、怎么生成最终回答。整个流程是开发者的代码在编排的：你的代码决定什么时候调用模型、什么时候执行工具、什么时候把结果塞回去。Agent 在 Function Calling 基础上多了“自主决策循环”——模型自己判断下一步该做什么、要不要继续调工具、什么时候该停下来给用户最终回答。区别的核心在于“谁在做编排”：Function Calling 是开发者编排，Agent 是模型自己编排。一个只有 Function Calling 能力的系统，可以被开发者编排成 Agent，但 Function Calling 本身不等于 Agent。

19.1.9 模型是怎么学会调用工具的

到这里你可能会有一个疑问：模型的这种“该用工具就用、不该用就不用、参数还能填对”的能力，是怎么来的？

答案是训练出来的，主要是在指令微调阶段。

训练数据长什么样呢？你可以想象这样一条样本：

用户：杭州明天会下雨吗？

助手（调用意图）：调用 `get_weather`，参数 `city="杭州"`

工具返回：`{"condition": "小雨", "temperature": 18}`

助手（最终回答）：杭州明天有小雨，气温 18 度，记得带伞。

训练的时候，模型需要学会的是中间那一步：看到“杭州明天会下雨吗”这个问题和工具清单之后，知道应该输出“调用`get_weather`，参数`city=杭州`”。

但你可能会有一个很自然的疑问：工具返回的结果（“小雨，18 度”）也在训练数据里，模型会不会死记硬背这个结果？将来不管谁问杭州天气，它都回答“小雨 18 度”？

不会。原因和前面讲指令微调时说过的 Loss Mask 机制是一样的。Loss Mask 的意思是：训练时只让某些位置参与损失计算，其他位置只作为上下文给模型看见，不推动参数更新。

放到这里，工具返回的结果本身不是模型要生成的目标，所以不会拿来训练模型背答案。模型真正要学的是两件事：第一，看到“杭州明天会下雨吗”时，知道该调用天气工具；第二，等工具真的返回“小雨，18 度”之后，能把这个结果转成一句自然的话回答用户。

也就是说，它学的是“什么时候该调工具、参数怎么填、拿到结果后怎么回答”这套模式，不是在没看到工具结果的时候，凭空背出某一天的天气。

训练数据里当然不可能覆盖所有的工具和所有的问题。模型能在没见过的工具上也表现不错，靠的是从大量训练样本中学会的泛化能力：它学会的是“看到实时数据类的问题 → 找到工具清单里最匹配的工具 → 提取用户问题里的关键信息填入参数”这套通用的行为模式，而不是死记硬背“看到天气问题就调`get_weather`”。

还有一个常见的现象你也会遇到：不同模型的 Function Calling 能力差异很大。有些模型几乎每次都能选对工具、填对参数；有些模型时不时就会犯错，比如该用工具的时候不用，或者参数填得一塌糊涂。这个差异的根源，一方面在于训练数据里工具调用样本的数量和质量不同，另一方面在于模型本身的推理能力不同。一般来说，参数量越大的模型，Function Calling 表现越好。

面试考点 · Function Calling 的训练数据怎么处理工具返回值

常见问法：“训练 Function Calling 能力时，工具返回的具体结果（比如‘22 度’）也在训练数据里。模型会不会把这些具体数值背下来？怎么防止？”

典型错误：答不上来，或者含糊地说“模型有泛化能力所以不会死记”。

正确思路：防止模型记住工具返回值的关键手段是 Loss Mask。在计算损失函数的时候，只对模型自己需要生成的部分（调用意图和最终回答）计算 loss。工具返回的结果在对话结构里通常被标记为一个独立角色（比如 tool 或 function），训练时这部分内容和 user 消息一样被 mask 掉，不参与梯度更新。底层机制和指令微调时只对 assistant 部分算 loss 是一样的，都是通过把不需要模型学习生成的位置标记为忽略来实现的。模型被训练去学的是“什么时候该调什么工具、参数怎么填”这个决策模式，而不是记住工具返回的具体数值。推理阶段的工具返回值是实时获取的，和训练数据里的具体数字无关。

19.1.10 额外提一个容易踩的坑：token 消耗

Function Calling 有一个成本上的细节，工作中经常被忽略：**每次工具调用至少需要两次模型推理**。第一次是模型输出调用意图，第二次是模型根据工具结果生成最终回答。如果是连续调用多个工具，模型可能被调用三次、四次甚至更多次。每次调用都要消耗 token 和费用。

另外，你传过去的工具清单本身也是占上下文窗口的。你声明了十个工具，每个工具的描述加参数可能有几十行，这些全部要算进 token 数量里。工具越多、描述越详细，每次请求的固定消耗就越高。

你挂了 15 个工具，每个工具的描述加参数占 200 个 token，光工具清单就吃掉了 3000 个 token。每次请求都要付这 3000 个 token 的钱，不管用户问的问题和工具有没有关系。一天 10 万次请求，一个月下来光工具清单就消耗了近百亿 token 的输入额度。就算单次看起来不贵，放到这个量级也会变成一笔很可观的固定成本。

所以在实际项目里，你不应该无脑地把所有能想到的工具都一股脑塞进去。只传当前场景下真正可能用到的工具，是一个值得养成的习惯。

19.1.11 自检清单

1. 大模型在 Function Calling 过程中到底做了什么？

它只做了文本生成。在需要工具的时候，它生成了一段结构化的调用意图（给程序看的，不是给用户看的），告诉程序“我想用这个工具，参数是这些”。真正执行工具的是你写的程序。

2. 一次完整的工具调用，模型被调用了几次？

至少两次。第一次输出调用意图，第二次根据工具返回的结果生成最终回答。中间执行工具的那一步是你的程序在做。

3. 模型的调用意图是输出给谁看的？

给你的后台程序看的，不是给用户看的。用户看不到这段结构化数据。只有模型根据工具结果生成的最终自然语言回答，才是展示给用户的。

4. 工具描述写得好不好，会影响什么？

直接影响模型选不选得对工具、填不填得对参数。工具描述本质上就是 Prompt，写得越清楚、边界越明确，模型的表现越好。

5. Function Calling 和 Agent 是什么关系？

Function Calling 是一次工具调用的标准协议：选工具、填参数、传结果、出回答。Agent 是在 Function Calling 基础上加了自主决策的循环：模型自己判断要调几次、调哪些、什么时候停。Function Calling 是 Agent 的基础组件，但两者不是同一件事。

19.1.12 Python 交互实验室

正文里走完了五步流程，但纯看文字，你可能还是很难在脑子里同时追踪四个参与者之间的消息流向：谁在第几步对谁说了什么、消息列表是怎么一步步变长的。配套的 `function_calling_demo.py` 把三个场景（数学计算、天气查询、闲聊）的完整流程逐步打印出来，每一步都标注了“这段输出是给程序看的”还是“给用户看的”。其中计算器工具是真正在用 Python 算数的。试试把参数配置区的数学表达式改成